Name: Youssef Tarek Isamil Hefny

ID: 231027091

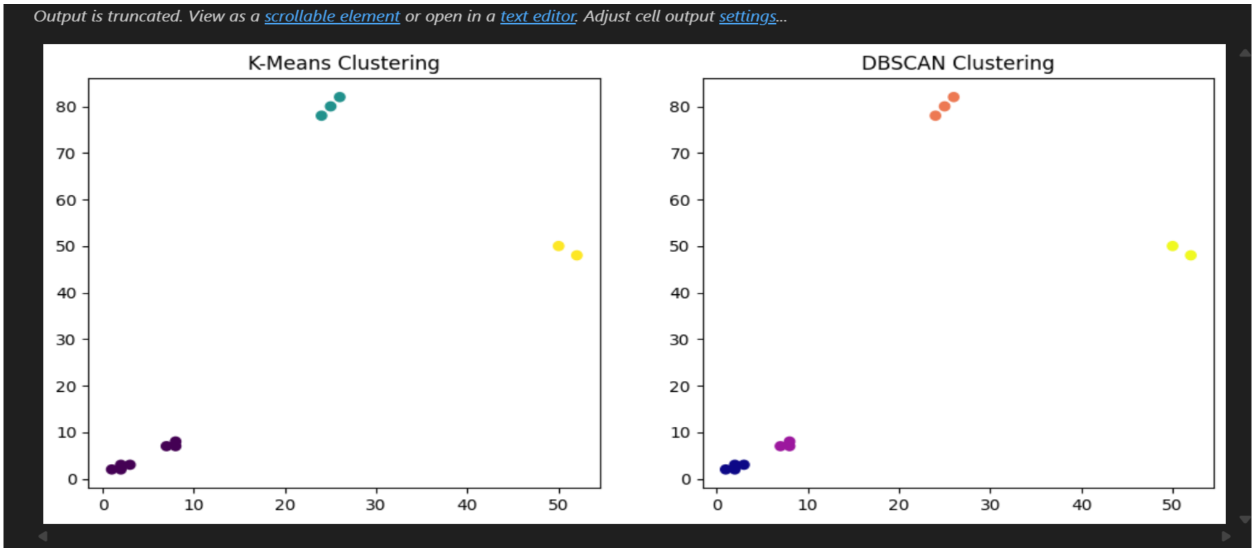
Assignment 6: Manual Clustering using K-Means and DBSCAN

Solution:

```python
import matplotlib.pyplot as plt

data = {
    'Point': ['P1', 'P2', 'P3', 'P4', 'P5', 'P6', 'P7', 'P8', 'P9', 'P10', 'P11', 'P12'],
    'X': [1, 2, 2, 3, 8, 8, 7, 25, 24, 26, 50, 52],
    'Y': [2, 2, 3, 3, 7, 8, 7, 80, 78, 82, 50, 48]
}

df = pd.DataFrame(data)
X_points = df[['X', 'Y']]

kmeans = KMeans(n_clusters=3, random_state=42)
df['KMeans_Cluster'] = kmeans.fit_predict(X_points)

print("--- K-Means Results ---")
print("Final Centroids:\n", kmeans.cluster_centers_)
print(df[['Point', 'X', 'Y', 'KMeans_Cluster']])
print("\n")

dbscan = DBSCAN(eps=3, min_samples=2)
df['DBSCAN_Cluster'] = dbscan.fit_predict(X_points)

print("--- DBSCAN Results ---")

print(df[['Point', 'X', 'Y', 'DBSCAN_Cluster']])
print("\n")

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(df['X'], df['Y'], c=df['KMeans_Cluster'], cmap='viridis')
plt.title('K-Means Clustering')

plt.subplot(1, 2, 2)
plt.scatter(df['X'], df['Y'], c=df['DBSCAN_Cluster'], cmap='plasma')
plt.title('DBSCAN Clustering')

plt.show()
```

17.0s

```
--- K-Means Results ---
Final Centroids:
 [[ 4.42857143  4.57142857]
 [25.         80.        ]
 [51.         49.        ]]
   Point   X   Y  KMeans_Cluster
0     P1   1   2               0
1     P2   2   2               0
2     P3   2   3               0
3     P4   3   3               0
4     P5   8   7               0
5     P6   8   8               0
6     P7   7   7               0
7     P8  25  80               1
8     P9  24  78               1
9    P10  26  82               1
10   P11  50  50               2
11   P12  52  48               2


--- DBSCAN Results ---
   Point   X   Y  DBSCAN_Cluster
0     P1   1   2               0
1     P2   2   2               0
2     P3   2   3               0
...
10   P11  50  50               3
11   P12  52  48               3
```



Part 3: Comparison & Reflection

1. Comparison between K-Means and DBSCAN:

- K-Means: It is a centroid-based algorithm that partitions the data into a fixed number of clusters (K=3). It works best when clusters are circular and similar in size.
- DBSCAN: It is a density-based algorithm that groups points based on how close they are to each other. It does not require a pre-defined number of clusters and can find clusters of any shape.

2. Handling of Outliers:

- K-Means: It is very sensitive to outliers. Points like P8, P9, and P10 (which are far from others) are forced into a cluster, which shifts the final centroids away from the actual centers of the main data.
- DBSCAN: It handles outliers much better. Since it relies on density, it can identify isolated points as "Noise" (labeled as -1) instead of forcing them into a cluster where they don't belong.

3. Which method performs better?

- In this specific dataset, DBSCAN performs better.
- Why? Because the dataset contains groups of points that are very far from each other (large gaps). DBSCAN correctly identifies these dense regions as separate clusters without being affected by the large distances, whereas K-Means struggles with the scale and the potential outliers.